Building a Catan Engine and Training It with Reinforcement Learning

A from-scratch, step-by-step build guide in Python

Engine • graph-based representation • PPO from scratch • self-play • MCTS / AlphaZero

---

This document is a roadmap you implement yourself. It explains *what* to build at each step, *why* it matters, the concepts behind it, the pitfalls to avoid, and pseudocode to guide you - without handing you the finished code. This edition goes deep on the part that decides whether your agent ever learns anything: the **representation** of state and actions. It also weighs whether **Monte Carlo Tree Search** is worth adding.

Table of Contents

0. How to Use This Guide

1. Project Setup and Structure

2. Understanding the Catan Board (Geometry)

3. Modeling Game State

4. The Game Engine and Rules

5. Validation and Testing

6. Representation I - Why Representation Decides Everything

7. Representation II - Encoding the State

8. Representation III - Graph Neural Networks for Catan

9. Representation IV - Symmetry and Data Augmentation

10. Representation V - The Action Space and Graph-Tied Policy Heads

11. Reinforcement Learning Foundations

12. Implementing PPO From Scratch

13. The Self-Play Training Loop

14. Monte Carlo Tree Search and AlphaZero for Catan

15. Evaluation and Diagnostics

16. Iteration and Reward Shaping

17. Expanding the Rules

18. Scaling Up (GPU / Colab)

A. Appendix: Reference Tables

B. Appendix: Pitfalls Checklist

C. Appendix: Glossary and Resources

0. How to Use This Guide

This guide breaks a large project into phases that each end with something you can run and verify. The single most important habit you can build is this: **never let two unverified systems depend on each other.** Get the game engine provably correct before you add a learning agent on top of it, because debugging a reinforcement learning agent that sits on top of a buggy simulator is one of the most painful experiences in software. If the rules are wrong, the agent will happily learn to exploit your bug instead of playing Catan.

There is, however, one idea that deserves your attention from day one even though you build it later: **representation**. How you present the game to a neural network - the shape of the observation, how actions are encoded, what structure the network can exploit - matters more than the choice of learning algorithm. A mediocre algorithm on a great representation will beat a great algorithm on a poor one. Sections 6 through 10 are the heart of this guide for that reason, and they are the longest.

The recommended order of construction is deliberately the reverse of how it is fun to think about the project. It is tempting to start with the neural network. Resist that. Build from the ground up:

1. A correct, fast **game engine** that random agents can play to completion.
2. A Gym-style **environment** that exposes a carefully designed representation: observations, action encodings, action masks, and rewards.
3. A from-scratch **PPO agent** in PyTorch, ideally with a graph-aware network.
4. A **self-play training loop** plus an evaluation harness.
5. An **iteration phase**: reward shaping, tuning, then expanding the rules.
6. Optionally, a **search layer** (MCTS / AlphaZero) on top of the trained network.

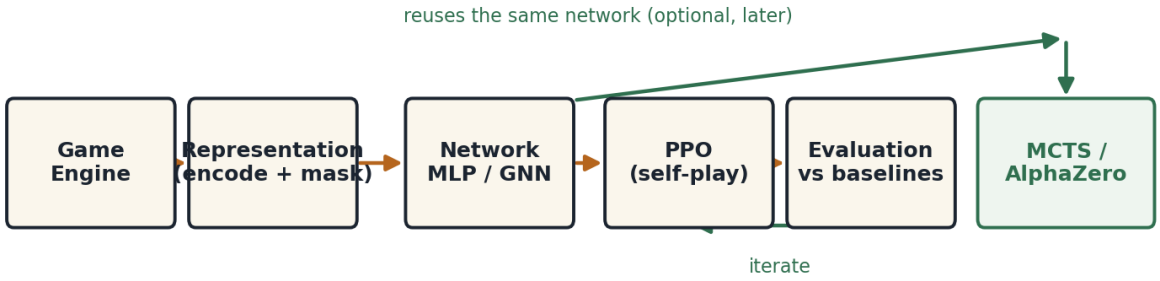


Figure 0.1 - The build pipeline. Each stage is verified before the next is added; MCTS/AlphaZero is an optional later layer that reuses the same trained network.

> **Tip** Each phase below ends with an explicit "Definition of done" checklist. Do not move to the next phase until every box is checked. This discipline will save you days.

Throughout, code samples are **pseudocode and skeletons** meant to convey structure and intent. You will write the real implementations. Type hints and docstrings are shown sparingly to keep snippets short; use them generously in your real code.

1. Project Setup and Structure

A clean package layout pays for itself immediately. It keeps the game rules separate from the learning code, which matters because you will rewrite the learning code many times while the engine stays stable. It also keeps the **representation** code in one place, so that when you change how the board is encoded (and you will, several times), you touch exactly one module.

1.1 Suggested layout

```
catan/
├── catan/                  # the game engine (pure Python, no torch)
│   ├── __init__.py
│   ├── board.py            # static geometry: tiles, vertices, edges, adjacency
│   ├── state.py            # mutable GameState
│   ├── actions.py          # Action types + the flat <-> structured action mapping
│   ├── engine.py           # rules: legal_actions / apply_action / terminal
│   └── players/
│       ├── __init__.py
│       └── random_agent.py # uniform-random legal-move agent (baseline)
├── rl/                     # everything that imports torch
│   ├── __init__.py
│   ├── encoding.py         # *** the representation layer: state -> tensors ***
│   ├── env.py              # Gym-style self-play wrapper
│   ├── network.py          # actor-critic network (MLP and/or GNN) with masking
│   ├── ppo.py              # PPO algorithm from scratch
│   ├── mcts.py             # (later) tree search guided by the network
│   ├── selfplay.py         # rollout collection + training loop
│   └── evaluate.py         # win-rate vs baselines
├── tests/                  # pytest unit tests for engine and encoding
├── scripts/                # one-off scripts (play a game, render a board)
├── requirements.txt
└── README.md
```

Note the dedicated rl/encoding.py. Putting all state-to-tensor logic behind one module with a stable interface (for example a function returning node features, edge features, global features, and the action mask) lets you swap a flat-MLP encoding for a graph encoding without disturbing the training loop.

1.2 Why separate catan/ from rl/

- The engine has **no machine-learning dependencies**. You can run thousands of random games in a tight loop without importing PyTorch. This keeps tests fast.
- You can reuse the engine for non-RL purposes: a rules-based bot, a web UI, a search-only agent, or a human-vs-AI script.
- When you later vectorize environments or add MCTS, only rl/ changes.

1.3 Dependencies

Keep the dependency list minimal at first. You truly only need a few things to start; add a graph library only when you actually build the GNN (Section 8).

```
# requirements.txt
numpy
torch
pytest

# add later, when you need them:
# torch_geometric   # OR dgl - graph neural networks (Section 8)
# matplotlib        # plotting training curves and debugging the board
# tqdm              # progress bars
# tensorboard       # richer training logs
```

Tip On a CPU-only laptop, install the CPU build of PyTorch to avoid pulling huge CUDA wheels you cannot use yet. You can hand-roll message passing in plain PyTorch if you want to avoid the heavier graph libraries entirely (Section 8.5).

Definition of done (Phase 1 setup)

- The folder structure exists and every package has an __init__.py.
- requirements.txt installs cleanly in a fresh virtual environment.
- import catan and import rl both succeed (even if mostly empty).

2. Understanding the Catan Board (Geometry)

Catan's board geometry is the foundation everything else stands on - and, as you will see in Sections 6 to 10, it is also the structure your representation should exploit. Get the adjacency relationships right and both the rules and the neural network become easy; get them wrong and you will chase impossible bugs for hours. The standard base-game board has a fixed shape, so you can compute its geometry once at import time and treat it as constant.

2.1 The three kinds of locations

Element	Count	What lives here
Hex tiles	19	A resource type and a number token (2-12, no 7)
Vertices (corners)	54	Settlements and cities
Edges (sides)	72	Roads

A vertex touches one, two, or three tiles. An edge connects exactly two vertices. These relationships are the heart of the rules and the skeleton of the graph the network will reason over: a settlement collects resources from every adjacent tile, and a road can only be built on an edge connected to your existing network.

19 hex tiles | 54 vertices (white dots) | 72 edges (tile sides)

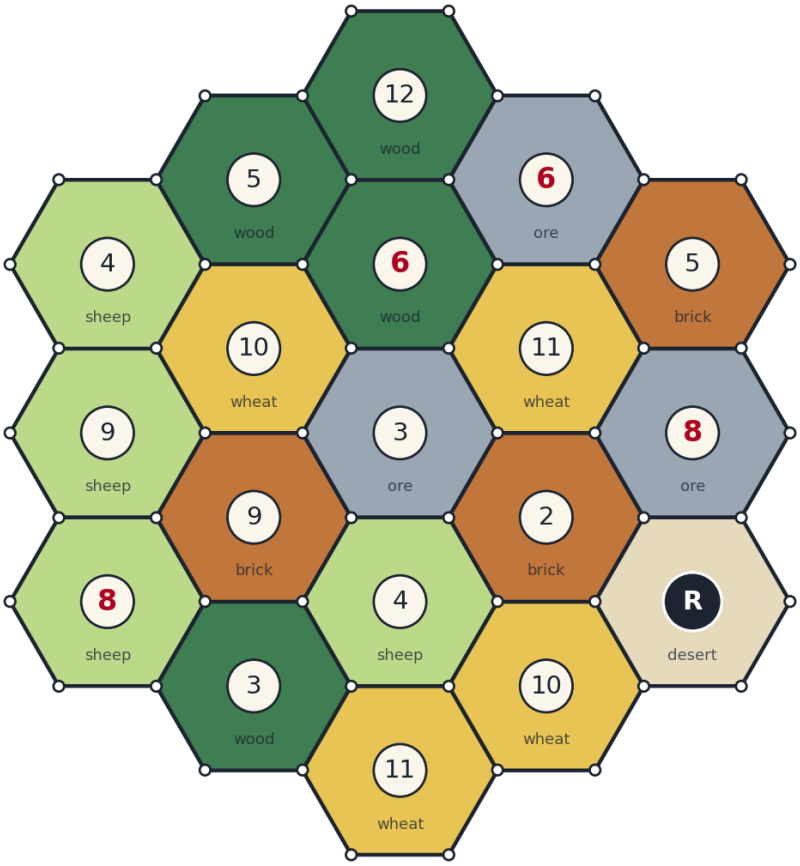


Figure 2.1 - A randomized base-game board. Tiles carry a resource (color) and a number token; red 6 and 8 are the most productive. White dots are the 54 vertices (settlement/city spots); tile sides are the 72 edges (road spots). The robber (R) starts on the desert.

2.2 Choose a coordinate system

You have two realistic options. Pick one and commit.

Option A - Hardcode the tables (pragmatic)

- Manually number the 19 tiles, 54 vertices, and 72 edges, then write out adjacency tables by hand or with a short generator script.
- Pro: no clever math; what you see is what you get. Con: tedious and error-prone, so you must validate it carefully (see 2.5).

Option B - Generate from hex coordinates (elegant)

- Use **axial** or **cube** coordinates for the hexes. Each hex has up to six neighbors found by adding fixed direction vectors.
- Derive vertices and edges from hex corners, deduplicating shared corners/sides.
- Pro: less manual data, naturally correct once the generator is right, and it gives you real coordinates you can reuse for rendering and for the symmetry transforms in Section 9. Con: the corner deduplication logic takes care to get right.

Tip Prefer Option B. The coordinates you derive are not just for drawing - they let you compute the board's rotational and reflective symmetries, which is the single most powerful data-augmentation trick available to you (Section 9).

2.3 The lookup tables you need

Aim to precompute and expose these structures (names are suggestions):

Table	Maps	Used by
vertex_tiles[v]	vertex -> list of adjacent tile ids	resource payout; node features
vertex_neighbors[v]	vertex -> list of adjacent vertex ids	distance rule; graph edges
vertex_edges[v]	vertex -> list of incident edge ids	road connectivity; graph edges
edge_vertices[e]	edge -> its 2 endpoint vertices	road placement; graph edges
tile_vertices[t]	tile -> its 6 corner vertices	payout; tile-node features
port_vertices[pt]	port -> the 1-2 vertices granting it	trade ratios (later)

Notice the right-hand column: every one of these tables does double duty, serving both the rules engine and the graph representation. That is not a coincidence - the graph the network reasons over *is* the board's adjacency structure.

2.4 Setting up a fresh board

Two things are randomized at the start of a game: which resource sits on each tile and which number token sits on each tile. The bags of resources and number tokens are fixed (see Appendix A). A standard setup also forbids the red numbers 6 and 8 from being adjacent, but you can ignore that refinement at first.

```
RESOURCE_BAG = (["wood"]*4 + ["sheep"]*4 + ["wheat"]*4
                + ["brick"]*3 + ["ore"]*3 + ["desert"]*1)   # 19 total
NUMBER_BAG = [2, 12] + [3,4,5,6,8,9,10,11]*2                # 18 tokens

def make_board(rng):
    resources = rng.permutation(RESOURCE_BAG)
    numbers = assign_numbers(resources, rng)   # the desert gets NO token
    robber_tile = index_of_desert(resources)   # robber starts on the desert
    return Board(resources, numbers, robber_tile)
```

2.5 Validate the geometry before trusting it

This is the cheapest hour you will ever spend. Write a tiny script that asserts your tables are internally consistent. If any of these fail, fix the geometry now.

- Every edge's two endpoints list that edge back in vertex_edges.
- Adjacency is symmetric: if v is a neighbor of w, then w is a neighbor of v.
- Counts are exactly 19 tiles, 54 vertices, 72 edges.
- Every vertex touches between 1 and 3 tiles; no more, no less.
- Render the board with matplotlib and eyeball it - seeing it once removes all doubt and also confirms your coordinates are right for later symmetry work.

Definition of done (Phase 2)

- All lookup tables exist and pass the consistency assertions above.
- You can create a randomized board with a seeded RNG and get reproducible output.
- You have real 2D coordinates for tiles, vertices, and edges (for rendering + symmetry).

3. Modeling Game State

The **board** (Section 2) is static for a whole game. The **state** is everything that changes: who owns what, how many resources each player holds, where the robber is, whose turn it is, and which phase of the turn we are in. Keep these two concepts firmly separated - the board feeds static features into the representation, the state feeds dynamic ones.

3.1 What the state must track

- **Per player**: a hand of the five resources (counts), victory points, and how many settlements/cities/roads they still have left to place (5 / 4 / 15).
- **Ownership arrays**: for each vertex, who owns it and whether it is a settlement or city; for each edge, who owns the road (or none).
- **Robber**: which tile it currently blocks.
- **Turn bookkeeping**: current player index, current phase, the last dice roll.
- **The bank**: how many of each resource remain in the supply (cards are finite).

3.2 Represent it for speed

Because the RL loop will create and read state millions of times - and MCTS, if you add it, will copy state thousands of times per move - prefer flat, cheap structures over deeply nested objects. Integer arrays indexed by vertex/edge id are ideal. Use a sentinel like -1 for "no owner". This layout also maps

directly onto the per-node and per-edge feature tensors of Section 7.

```
class GameState:
    def __init__(self, board, n_players=4, rng=None):
        self.board = board
        self.n_players = n_players
        self.rng = rng

        self.hands = zeros((n_players, 5), dtype=int)         # resource counts
        self.vp = zeros(n_players, dtype=int)
        self.pieces_left = full((n_players, 3), [5, 4, 15])    # S, C, R

        self.vertex_owner    = full(54, -1, dtype=int)         # -1 = none
        self.vertex_building = zeros(54, dtype=int)            # 0/1/2
        self.edge_owner      = full(72, -1, dtype=int)

        self.robber_tile = board.robber_tile
        self.current_player = 0
        self.phase = Phase.SETUP_SETTLEMENT
        self.last_roll = None
        self.bank = full(5, 19, dtype=int)
```

3.3 Copy vs mutate - decide early

Reinforcement learning loops simulate forward constantly, and tree search multiplies this enormously. You need a clear policy on whether apply_action mutates the state in place or returns a new copy.

- **Functional copies**: apply_action returns a fresh state. Simpler to reason about and to test; copy cost is acceptable if the state is flat numpy arrays. Start here.
- **Mutate + undo**: fastest, essential for deep MCTS, but you must be disciplined about saving/restoring. Switch to this only after profiling shows copying dominates.

3.4 Hidden information matters for representation and search

In real Catan, opponents' hands are hidden, the development-card deck order is unknown, and the robber steals a random card. In the minimal version with no trading and no dev cards, the only hidden element is the random steal. **Decide deliberately** how much the acting agent observes, because this choice ripples through both the representation (Section 7) and any tree search (Section 14, where hidden information forces you toward determinization or information-set MCTS).

Tip Recommended starting point: full observability for self-play (the agent sees true hands). It simplifies everything and lets you validate learning. Tighten to realistic partial observability later, and only then worry about its consequences for search.

Definition of done (Phase 3)

- A GameState you can create, deep-copy, and compare for equality.
- Every quantity above is represented and initialized correctly for a new game.

4. The Game Engine and Rules

This is the largest engine component. It answers two questions and nothing more: **what can the current player legally do right now?** and **what happens when they do a specific thing?** Everything else - agents, training, search, UI - is built on top of those two functions.

4.1 The two core functions

```
def legal_actions(state) -> list[Action]:
    '''All actions the current player may legally take in the current phase.'''

def apply_action(state, action) -> GameState:
    '''Return the resulting state after performing `action`.
       Must be deterministic given the state's seeded RNG.'''
```

If these two functions are correct and total (they never crash and never allow an illegal move), the rest of the project is downhill. The action mask in Section 10 is literally legal_actions projected onto a fixed index space, and a tree search expands nodes by calling exactly these two functions - so their correctness is load-bearing for the entire stack.

4.2 Turn phases

Model the turn as an explicit state machine. Being explicit about phase prevents a whole category of bugs, and it gives the representation a clean categorical feature ("which decision am I being asked to make?").

```
class Phase(Enum):
    SETUP_SETTLEMENT = 1   # place a settlement (initial placement)
    SETUP_ROAD       = 2   # place a road touching that settlement
    ROLL             = 3   # roll the dice
    ROBBER_MOVE      = 4   # only entered after rolling a 7
    ROBBER_STEAL     = 5   # choose a victim adjacent to the new robber tile
    MAIN             = 6   # build / trade / end turn
    GAME_OVER        = 7
```

The setup phase (snake order)

Setup is its own mini-game and a common source of bugs. Players place in order 0,1,2,3 and then back 3,2,1,0. Each placement is a settlement plus an adjacent road. The settlement placed during the *second* round grants its owner one resource for each adjacent non-desert tile. The distance rule still applies during setup, but roads do not need to connect to an existing network the way they do later. Setup decisions are disproportionately important to the final result, which is exactly why a search layer (Section 14) can pay off most here.

The main phase

1. **Roll** two dice. On any number other than 7, every tile with that number pays its adjacent settlements (1 card) and cities (2 cards), drawn from the bank.

2. **On a 7**: any player holding more than 7 cards discards half (round down), then the current player moves the robber to a new tile and steals one random card from a player with a building on that tile.
3. **Build / trade**: spend resources to build roads, settlements, or cities, or do a 4:1 trade with the bank. The player may take any number of legal build/trade actions.
4. **End turn**: pass control to the next player.

4.3 Building rules in detail

Build	Cost	Key constraints
Road	1 wood + 1 brick	Empty edge connected to your road/building network
Settlement	wood+brick+sheep+wheat	Empty vertex, connected to your road, obeys distance rule
City	2 wheat + 3 ore	Upgrades one of your own existing settlements

The **distance rule**: no settlement may be placed on a vertex adjacent to any occupied vertex. The **connectivity rule**: after setup, a new road must touch one of your existing roads, settlements, or cities, and a new settlement must sit on a vertex touched by one of your roads.

4.4 Resource distribution and the bank

- If a roll would pay out more of a resource than the bank holds and **more than one player** is owed it, officially **nobody** gets that resource; if only one player is owed it, they take whatever remains. Simplify at first, but document the deviation.
- Cities pay **two** cards, settlements pay **one**. The robber's tile pays nothing while the robber sits on it.

4.5 Winning, determinism, and seeding

A player wins immediately upon reaching 10 victory points on their own turn. Route **all** randomness - board setup, dice, robber steal - through a single seeded generator on the state so games are reproducible. Reproducibility is not a nicety: it lets you replay the exact game where the agent did something strange, and it is required for the determinization tricks in Section 14.

Definition of done (Phase 4)

- legal_actions never returns an illegal move in any phase.
- apply_action correctly updates resources, ownership, VP, bank, and phase.
- A full game can be played end to end by calling these two functions in a loop.
- Given the same seed, a game replays identically every time.

5. Validation and Testing

Before a single neural network appears, prove the engine works. This phase buys peace of mind for the entire rest of the project.

5.1 The random-agent smoke test

```
def play_random_game(seed):
    state = new_game(seed)
    while not is_terminal(state):
        actions = legal_actions(state)
        assert actions, "no legal actions but game not over!"
        state = apply_action(state, rng.choice(actions))
        assert_invariants(state)
    return winner(state)

for seed in range(10_000):
    play_random_game(seed)
```

5.2 Invariants to assert continuously

- Total cards of each resource (all hands + bank) is conserved at 19 (until dev cards/ports change supply assumptions).
- No player's piece counts go negative; placed pieces + remaining = starting count.
- Victory points equal settlements + 2 x cities (in the minimal game).
- No two settlements/cities violate the distance rule; every road connects (after setup).
- Games always terminate (cap turns to catch infinite loops while debugging).

5.3 Targeted unit tests

Smoke tests catch crashes; unit tests catch *wrong* behavior that does not crash. Write small, deterministic pytest cases for the rules most likely to be subtly wrong: the distance rule, road connectivity, a roll of 8 paying the right cards (double for cities), a 7 triggering discards and a working robber steal, bank depletion, and exact resource deduction on building.

Definition of done (Phase 5)

- 10,000+ random games run with zero crashes and zero invariant violations.
- A unit-test suite covers the tricky rules and passes.
- You trust the engine enough to debug the agent *through* it, not *around* it.

6. Representation I - Why Representation Decides Everything

This is the most important conceptual section in the guide. A reinforcement learning agent never sees "Catan." It sees whatever numbers you choose to feed it, and it produces whatever action indices you choose to define. That translation - state to numbers, network output to moves - is the **representation**, and it sets a hard ceiling on how well any algorithm can possibly do. You can change PPO for something fancier and gain a little; you can fix a bad representation and gain a lot.

6.1 What a good representation must achieve

- **Completeness**: it contains everything needed to act well. If a fact the agent needs is missing, no amount of training can recover it.
- **Fixed shape**: networks need fixed-size inputs and outputs even though Catan's legal-move count changes every step. (Solved by masking - Section 10.)
- **Locality / structure**: facts that are related in the game (a tile and its corners) should be related in the representation, so the network does not have to rediscover the board's geometry from scratch.
- **Invariance**: the agent should treat equivalent situations equivalently - the same position viewed from a different seat, or the board rotated, should not look like a brand-new problem.
- **Good numerical conditioning**: inputs on similar, modest scales train far faster.

6.2 The two big invariances you must respect

Player (seat) invariance

Catan is symmetric in its players: the rules do not care whether you are seat 0 or seat 2. If you encode the board in absolute seat terms, the network must separately learn the same strategy four times, once per seat. Instead, encode everything **relative to the player about to act**: "my" buildings always occupy the same channel, "the opponent to my left" the next, and so on. One shared policy then plays every seat, and every game yields four times as much useful experience.

Board (spatial) symmetry

The Catan board has the symmetry of a hexagon: six rotations and six reflections, twelve in total. A board rotated 60 degrees is a different-looking but strategically identical position. You can exploit this for massive data augmentation (Section 9) and, with the right network, for built-in invariance. This is the closest thing to a free lunch in the whole project.

6.3 The spectrum of representations

There is a progression from crude to powerful. You do not have to build the most powerful one first - but you should know where you are on this ladder and why.

Representation	Network	Pros / cons
Flat feature vector	MLP	Simplest; ignores board structure; weakest ceiling
Fixed multi-channel tensor	MLP / hex-CNN	Keeps some locality; awkward on hex grids

Graph (nodes + edges)	GNN	Matches the board exactly; strongest; more to build

Recommended path: **start with a well-designed flat vector + MLP** to validate the whole pipeline quickly (Section 7), then **upgrade to a graph neural network** (Section 8) once everything works. The flat encoding is not throwaway - it is your correctness baseline and a useful ablation.

Definition of done (Phase 6)

- You can articulate, in one sentence each, completeness, fixed shape, structure, invariance, and conditioning for your chosen encoding.
- You have decided on relative-to-current-player encoding as a non-negotiable.

7. Representation II - Encoding the State

Here you turn a GameState into tensors. Even if you go straight to a graph network, you still need to decide *which features* live on each tile, vertex, edge, and on the global context. Design the features first; the network architecture consumes them.

7.1 Group features by where they live

Catan has four natural feature groups. Designing them separately keeps the encoding clean and makes the later jump to a graph network almost mechanical.

Group	Count	Example features (per element)
Tile features	19	resource one-hot (6), roll-probability (1), robber flag (1)
Vertex features	54	owner-relative one-hot (5), building type (3), port access (n)
Edge features	72	road owner-relative one-hot (5)
Global features	1	my hand (5), opp hand sizes, all VPs (rotated), bank (5), phase one-hot, last roll

7.2 Encode discrete things sensibly

- **One-hot** small categoricals (resource type, building type, phase). Avoid feeding a raw integer like resource=3; the network would wrongly assume ore is "more than" sheep.
- **Owner channels are relative**: index 0 = me, 1 = next player clockwise, etc., with a separate "unowned" channel. Never use absolute seat numbers.
- **Number tokens as probability**: represent a tile's number by its dice probability (6 and 8 are large, 2 and 12 tiny) rather than its face value. This hands the network the one fact that actually matters about the token.
- **Embeddings** are an option for resource type if you later want learned category vectors, but one-hot is perfectly fine to start.

7.3 Normalize the continuous things

Neural networks train far better when inputs share a modest scale, roughly zero-centered with unit-ish variance. Divide card counts by a sensible constant (say 10), express VP as a fraction of the 10 needed to win, and keep flags as 0/1. A single oversized raw feature can dominate early gradients and stall learning.

7.4 The flat-vector encoding (your first version)

For the MLP baseline, concatenate the four groups into one long vector. Keep the layout in a single function so the network's input dimension is derived, never hardcoded twice.

```
def encode_flat(state) -> np.ndarray:
    me = state.current_player
    tiles  = encode_tiles(state)            # (19, F_tile)  -> flatten
    verts  = encode_vertices(state, me)     # (54, F_vert)  -> flatten, relative
    edges  = encode_edges(state, me)        # (72, F_edge)  -> flatten, relative
    glob   = encode_global(state, me)       # (F_global,)   -> rotated to 'me' first
    return concatenate([tiles.ravel(), verts.ravel(),
                        edges.ravel(), glob])

OBS_DIM = len(encode_flat(sample_state))     # derive once, assert it is constant
```

Tip Always rotate per-player global features (VPs, hand sizes) so that the current player's own value comes first, then opponents in fixed clockwise order. This is the scalar-level version of relative encoding and is easy to forget.

7.5 Partial observability (when you add it)

Under realistic rules you cannot see opponents' exact hands. Then encode only what the acting player truly knows: own hand exactly, opponents by hand *size*, plus public history (e.g. cards revealed by trades or monopolies later). Keep a clean boundary between the true state (engine) and the observation (encoding), so the agent can never accidentally peek. This boundary is also what makes information-set search possible (Section 14).

Definition of done (Phase 7)

- A single encode_* module produces fixed-length, relative, normalized tensors.
- A unit test asserts the observation length is constant across many random states.
- Two states that are identical up to seat rotation encode to the same "my" channels.

8. Representation III - Graph Neural Networks for Catan

Catan is, structurally, a graph: vertices joined by edges, with tiles attached to their corners. A graph neural network (GNN) consumes exactly that structure, which is why it is the most effective

representation for this game. Where an MLP on a flat vector must learn from scratch that vertex 17 is next to vertex 18, a GNN is *told* the adjacency and spends its capacity on strategy instead of geography.

8.1 The core idea of message passing

A GNN repeatedly lets each node update its feature vector by mixing in messages from its neighbors. After one round, every node "knows" about its immediate surroundings; after k rounds, about everything within k hops. For Catan, a handful of rounds lets a vertex's embedding reflect its tiles, its roads, nearby settlements, and the robber - precisely the local context that determines how good that spot is.

```
# one message-passing layer, conceptually
for each node v:
    msg[v] = aggregate( transform(h[u]) for u in neighbors(v) )   # sum/mean/max
    h[v]   = update( h[v], msg[v] )                               # e.g. MLP + ReLU
# stack 2-4 such layers
```

8.2 Modeling Catan as a graph

You have a choice of how literally to mirror the board. Two good options:

- **Vertex graph (simplest effective choice)**: nodes are the 54 vertices; graph edges are the 72 board edges. Attach each vertex's incident tiles' features (resource, probability, robber) to that vertex's input features. Roads are edge features.
- **Heterogeneous graph (most faithful)**: three node types - tiles, vertices, edges - connected by their real incidences. Message passing flows tile -> vertex -> edge and back. More expressive, more to implement.

Tip Start with the vertex graph. It captures the spatial structure that matters with the least machinery, and it makes the graph-tied policy heads in Section 10 trivial.

8.3 Node, edge, and global features in the graph

These are exactly the feature groups from Section 7, now attached to graph elements instead of flattened. Tile information is folded into the vertices it touches (summed or concatenated). Global features (your hand, the bank, the phase) are broadcast to every node or handled by a separate global token that exchanges messages with all nodes.

8.4 Reading out the graph

After message passing you have a learned embedding per node. You read out in two ways, both of which you will need:

- **Per-node outputs** for spatial actions: the logit for "build a settlement at vertex v" is a small linear layer applied to v's embedding (Section 10). This couples the action space to the representation and shares weights across all locations.

- **A pooled graph embedding** for the value head and global actions: aggregate all node embeddings (mean/max/attention) into one vector summarizing the whole position.

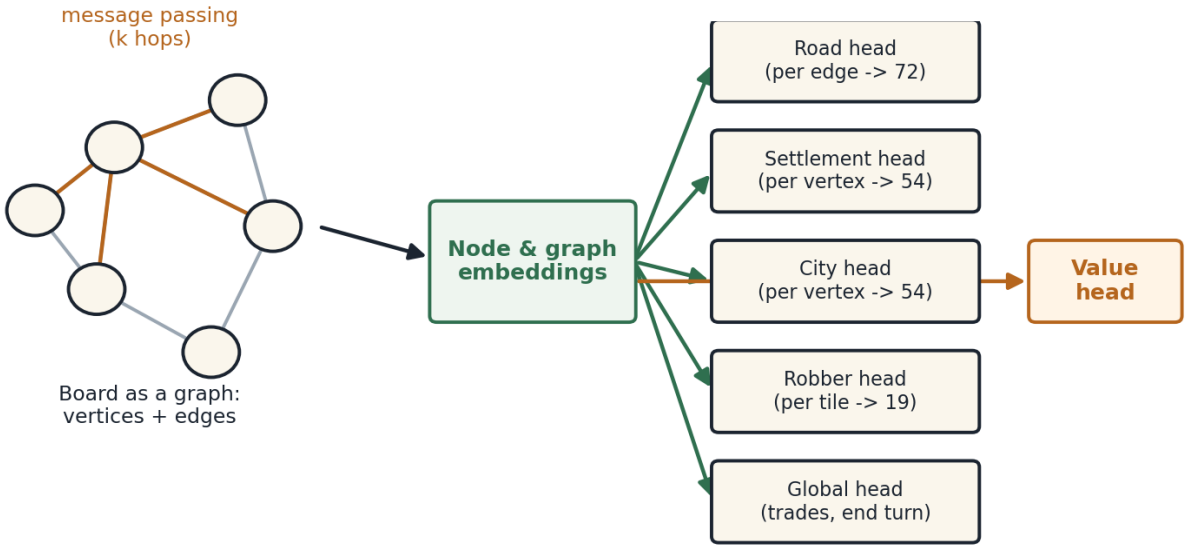


Figure 8.1 - The graph network. Message passing turns the board graph into per-node embeddings; factored policy heads read logits directly off each edge/vertex/tile, while a pooled embedding feeds the value head. Weights are shared across all locations of a kind.

8.5 Implementation options

- **PyTorch Geometric** or **DGL**: batteries-included message passing, batching of many graphs, and standard layers (GCN, GraphSAGE, GAT). Easiest if you are comfortable adding the dependency.
- **Hand-rolled**: because Catan's graph is small and fixed, you can implement message passing with a precomputed adjacency matrix and a couple of matrix multiplies in plain PyTorch. This keeps dependencies minimal and is very fast on CPU for a 54-node graph.

```
# hand-rolled layer for a fixed graph (A = normalized 54x54 adjacency)
H = relu(A @ (X @ W1))      # neighbor mixing
H = relu(A @ (H @ W2))      # second hop
node_emb = H                # (54, d)
graph_emb = node_emb.mean(0)  # plus max-pool, concatenated, often helps
```

8.6 Is the GNN worth it on a laptop?

Yes, eventually - but sequence it correctly. The GNN's 54-node graph is tiny, so the per-forward cost is modest even on CPU; the bottleneck in your training will be the Python game simulator, not the network (Section 18). Get the MLP pipeline learning first to prove your reward, masking, and loop are correct, then switch network.py to the GNN and expect a meaningful jump in sample efficiency and final strength.

Definition of done (Phase 8)

- An adjacency structure derived from the board's geometry tables.
- A GNN that maps (node features, edges, globals) to per-node embeddings + a graph embedding, with no NaNs.
- A documented switch in network.py between the MLP and GNN encoders.

9. Representation IV - Symmetry and Data Augmentation

This short section describes the highest-leverage trick available to you. The Catan board has twelve symmetries (six rotations and six reflections of the hexagon). Any permutation of tiles, vertices, and edges induced by one of these symmetries produces a position that is strategically identical to the original. You can use that fact two ways.

9.1 Data augmentation

Each time you store a training example (a state, the action taken, the outcome), you can also store up to eleven symmetric copies: apply a board symmetry to permute the node/edge features *and* the corresponding action index. This multiplies your data by up to 12x for free, which is enormous when self-play is expensive on a CPU. It also pressures the network toward genuine spatial invariance rather than memorizing one orientation.

```
# precompute, once: for each of the 12 symmetries, the permutation it
# applies to vertices, edges, and tiles (derived from the coordinates).
def augment(example):
    out = [example]
    for perm in BOARD_SYMMETRIES[1:]:        # skip identity
        out.append(relabel(example, perm))   # permute features AND action index
    return out
```

Tip Combine board symmetry with seat rotation (Section 6.2). Seat rotation gives 4x; board symmetry up to 12x. They are independent, so the multiplier compounds - though in practice you sample a subset per step rather than always expanding fully.

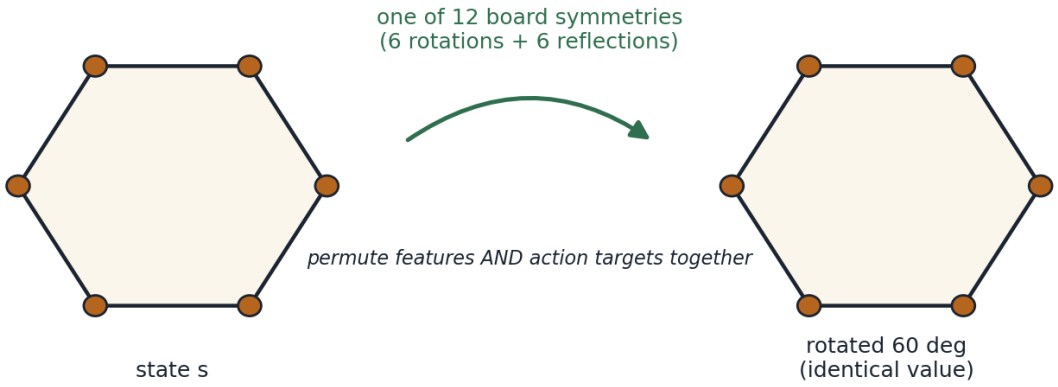


Figure 9.1 - Board symmetry. Rotating or reflecting the board yields a strategically identical position. Relabel the node/edge features and the action target together to turn one experience into up to twelve.

9.2 Getting the action permutation right

The subtle part: when you rotate the board, an action like "build a road on edge 12" becomes "build a road on edge perm[12]". Your augmentation must permute the **action target** consistently with the features, or you will train the network on mislabeled data - worse than no augmentation. Unit-test this by augmenting a state, applying the (permuted) action in both, and checking the resulting positions are themselves symmetric.

9.3 Built-in invariance (advanced)

Beyond augmentation, a GNN is already permutation-equivariant with respect to the graph: relabeling nodes relabels outputs the same way, automatically. That gives you much of the benefit structurally. Fully symmetry-equivariant architectures exist but are an advanced topic; augmentation captures most of the practical gain with far less complexity.

Definition of done (Phase 9)

- The twelve board symmetries are precomputed as permutations of tiles/vertices/edges.
- An augmentation function permutes features and the action target consistently, with a passing round-trip test.

10. Representation V - The Action Space and Graph-Tied Policy Heads

Representation is not only about inputs. How you represent **actions** - the network's outputs - is just as consequential, and Catan's large, variable action set is where many projects stumble.

10.1 The challenge: a variable action space

Unlike CartPole's two fixed actions, Catan's legal moves change every step: sometimes 30 edges are buildable, sometimes none; sometimes you must move the robber; sometimes you can only end your turn. A network needs a **fixed-size output**, so you cannot simply emit "the legal moves." The universal solution is a **fixed action space plus action masking**: enumerate every action that could ever be legal, then each step supply a boolean mask of which are legal now.

10.2 Option A - one flat index space

Assign a contiguous block of indices to each action category. Simple, works with any network including the MLP baseline.

Index block	Size	Meaning
0 .. 71	72	Build a road on edge e (also setup roads)
72 .. 125	54	Build a settlement on vertex v (also setup settlements)
126 .. 179	54	Build a city on vertex v
180 .. 198	19	Move the robber to tile t
199 .. 218	20	Bank trade: give resource i, receive resource j
219	1	End turn / pass

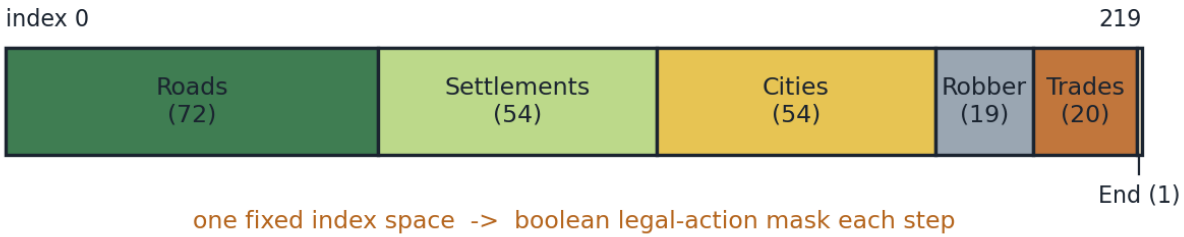


Figure 10.1 - One flat action index space. Every possible action gets a fixed index; a boolean mask marks which are legal on the current step.

Define this mapping in exactly one place (actions.py) with helpers to convert between an action and its index. The engine, the mask builder, and the network must all agree on it; a mismatch trains the agent on nonsense.

10.3 Option B - graph-tied (factored) policy heads

This is the more effective representation and the natural partner to the GNN. Instead of one undifferentiated vector of 220 logits, give the policy **structured heads** that read directly off the graph embeddings:

- **Road head**: a shared linear layer applied to each edge's embedding -> 72 logits.

- **Settlement head** and **city head**: a shared linear layer applied to each vertex's embedding -> 54 logits each.
- **Robber head**: applied to each tile's embedding -> 19 logits.
- **Global head**: from the pooled graph embedding -> the handful of non-spatial actions (bank trades, end turn).

Because the spatial heads **share weights across all locations**, the network learns "what makes a good settlement spot" once and applies it everywhere, instead of learning 54 independent output weights. This generalizes far better and is exactly how AlphaZero-style policies handle board games. You still concatenate all heads into one logit vector and apply the same mask - downstream code does not change.

Tip Factored heads also make symmetry augmentation cleaner: permuting node order permutes the per-node logits in lockstep, so the action relabeling of Section 9 falls out naturally.

10.4 Multi-part actions (the robber, trades, future trading)

Some decisions are naturally multi-step. Handle them as sequential phases rather than exploding the action space:

- **Robber**: "move robber to tile t" transitions into a ROBBER_STEAL phase whose legal actions are the steal targets. Two clean decisions, small action space.
- **Player trading (later)**: do not enumerate every possible offer. Use a phased propose/accept/reject protocol, or restrict to a small menu of canonical offers at first. Open-ended negotiation is the single biggest action-space blow-up in Catan.

10.5 Building and applying the mask

```
def action_mask(state) -> np.ndarray:        # shape (ACTION_DIM,), bool
    mask = zeros(ACTION_DIM, dtype=bool)
    for action in legal_actions(state):
        mask[action_to_index(action)] = True
    return mask

# in the network, BEFORE softmax:
logits = logits.masked_fill(~mask, -1e9)     # never after softmax!
```

Tip If the mask is ever all-False you have a bug: every turn the current player has at least "end turn" available. Assert mask.any() every step during development. Masking with -1e9 (not -inf) avoids NaNs.

Definition of done (Phase 10)

- A fixed ACTION_DIM with a bijective action<->index mapping, unit tested.
- action_mask agrees exactly with legal_actions on random states.
- (If using a GNN) factored policy heads that assemble into the same masked logit vector.

11. Reinforcement Learning Foundations

With representation settled, make sure the core RL ideas are solid before writing PPO. You do not need heavy math, but you need correct intuition, because PPO is just a careful recipe built from these pieces.

11.1 The vocabulary

Term	Meaning in Catan terms
Policy (pi)	The network mapping an observation to a probability over actions
Value V(s)	Estimated future reward from a state (how good is my position?)
Return	The actual (discounted) reward collected from a state onward
Advantage	How much better an action did than the value baseline expected
Trajectory	One game's sequence of (obs, action, reward) for a seat
Discount (gamma)	How much future reward is worth now (e.g. 0.99)

11.2 Actor-critic in one paragraph

You train two things at once that share a network body (your MLP or GNN encoder). The **actor** (policy head) decides what to do; the **critic** (value head) predicts how good the current state is. The critic's prediction is a baseline so the actor learns from the *advantage* - whether an action did better or worse than expected - rather than from raw, noisy returns. This reduces variance and is why actor-critic methods learn faster than vanilla policy gradients.

11.3 Why PPO specifically

- **Stable**: a clipped objective stops the policy from changing too much per update, avoiding the collapses that plague simpler methods.
- **Simple and on-policy**: collect fresh experience, do a few gradient epochs, discard, repeat.
- **Plays well with masked discrete actions** - exactly Catan's situation.
- **Ubiquitous**, so references and debugging help abound.

11.4 Credit assignment in a multi-agent game

Catan has four seats but (at first) one shared policy. Treat each seat's experience as its own trajectory: collect the (obs, action, reward) stream that seat saw and assign the terminal win/loss reward to it. Pool all four trajectories into one training batch. This is the simplest correct self-play and where you should start.

11.5 GAE - smoothing the advantage

Generalized Advantage Estimation trades bias against variance with one knob, lambda. Implement its short backward recurrence (Section 12) and remember: higher lambda means lower bias, higher variance. Common values: gamma=0.99, lambda=0.95.

12. Implementing PPO From Scratch

PPO is a surprisingly compact algorithm. Build it in three pieces: the network, the rollout buffer, and the update step. Whether the encoder is an MLP or a GNN, everything below is identical - that is the payoff of isolating the representation.

12.1 The actor-critic network

```
class ActorCritic(nn.Module):
    def __init__(self, encoder, action_dim):
        super().__init__()
        self.encoder = encoder           # MLP body OR GNN producing embeddings
        self.policy_head = PolicyHeads(...)   # flat (10.2) or factored (10.3)
        self.value_head  = nn.Linear(encoder.out_dim, 1)

    def forward(self, obs, mask):
        feats = self.encoder(obs)
        logits = self.policy_head(feats)
        logits = logits.masked_fill(~mask, -1e9)   # mask BEFORE softmax
        value  = self.value_head(pool(feats)).squeeze(-1)
        return logits, value
```

12.2 Sampling an action

```
dist = Categorical(logits=logits)
action = dist.sample()
logprob = dist.log_prob(action)
# store: obs, mask, action, logprob, value, reward, done
```

12.3 The rollout buffer

PPO is on-policy: collect a fixed batch of experience with the current policy (e.g. 4,096-8,192 steps across many games), learn from it, then discard it. The buffer just accumulates the per-step tensors above.

12.4 Returns and advantages with GAE

```
adv = 0.0
for t in reversed(range(T)):
    nonterminal = 1.0 - done[t]
    delta = reward[t] + gamma * value[t+1] * nonterminal - value[t]
    adv = delta + gamma * lam * nonterminal * adv
    advantages[t] = adv
returns = advantages + values[:T]
advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
```

12.5 The clipped objective

```
ratio = exp(new_logprob - old_logprob)
surr1 = ratio * advantages
surr2 = clip(ratio, 1 - eps, 1 + eps) * advantages
policy_loss = -min(surr1, surr2).mean()
value_loss  = mse(new_value, returns)
entropy     = dist.entropy().mean()
loss = policy_loss + c_v * value_loss - c_e * entropy
```

The three terms each do a job: the policy loss improves the actor, the value loss trains the critic, and the entropy bonus keeps the policy from collapsing to one action too early. Typical coefficients: value c_v=0.5, entropy c_e=0.01.

12.6 The update loop

1. Collect a batch with the current policy.
2. Compute advantages and returns with GAE.
3. For a few epochs (e.g. 4), shuffle into minibatches and take Adam steps, **re-evaluating** the network so the ratio reflects updated weights.
4. Optionally clip the gradient norm (e.g. 0.5). Discard the batch and repeat.

12.7 Recommended starting hyperparameters

Hyperparameter	Start	Notes
gamma (discount)	0.99	Catan games are long; keep it high
lambda (GAE)	0.95	Standard
clip epsilon	0.2	Lower if updates look unstable
learning rate	3e-4	Adam; anneal toward 0 over training
epochs per batch	4	More reuses data harder but risks overfitting
batch size (steps)	4096	Bigger = more stable, slower per update
minibatch size	256-512	Fits CPU memory easily
entropy coef	0.01	Raise if the policy collapses too fast
value coef	0.5	Rarely needs tuning

Definition of done (Phase 12)

- The network returns masked logits + value without NaNs for both encoders.
- GAE on a tiny synthetic trajectory matches a hand calculation.
- PPO solves a toy environment (e.g. CartPole) before you trust it on Catan.

13. The Self-Play Training Loop

Connect the environment and agent. The loop alternates between **acting** (playing many games to fill the buffer) and **learning** (the PPO update). The care is in collecting per-seat trajectories correctly.

```
policy = ActorCritic(encoder, ACTION_DIM)
opt = Adam(policy.parameters(), lr=3e-4)
for iteration in range(N):
    buffer = []
    while buffer.steps < BATCH_STEPS:                # ---- ACT ----
        env.reset(seed=next_seed())
        per_seat = defaultdict(list)
        obs, mask = env.observe(); done = False
        while not done:
            seat = env.current_player
            logits, value = policy(obs, mask)
            action, logprob = sample(logits)
            nobs, nmask, reward, done, _ = env.step(action)
            per_seat[seat].append(Transition(obs, mask, action, logprob, value))
            obs, mask = nobs, nmask
        assign_terminal_rewards(per_seat, winner=env.winner())
        buffer.extend(augment_with_symmetry(flatten(per_seat)))   # Section 9
    advantages, returns = gae(buffer, gamma, lam)    # ---- LEARN ----
    ppo_update(policy, opt, buffer, advantages, returns)
    log_metrics(iteration)
    if iteration % SAVE_EVERY == 0: save_checkpoint(policy, iteration)
```

13.1 Assigning rewards per seat

Each seat saw a different stream. On game end, give the winner's trajectory +1 and the losers' -1 (0 along the way, until you add shaping). Only the final transition of each seat carries the terminal reward; mark it done=True so GAE stops there.

13.2 Bootstrapping at truncation

Catan episodes always terminate, so post-terminal value is 0. If you truncate long games for speed, **bootstrap** with the critic's value at the cut point instead of treating it as a true loss - otherwise you teach the agent that running long is catastrophic.

13.3 Opponent snapshots (stabilizing self-play)

Training purely against your current self can oscillate or collapse. Periodically freeze a copy of the policy into a pool of past opponents, and sometimes seat the learner against these frozen snapshots. This 'league' keeps a memory of older strategies and prevents catastrophic forgetting. It is also the bridge to AlphaZero-style training in Section 14.

13.4 What to log every iteration

- Win rate vs a random baseline (headline metric).

- Average game length in turns (collapses/explosions signal trouble).
- Policy entropy (should decline gradually; a sudden crash means premature convergence).
- Value loss and the critic's explained variance.
- Mean episode reward and the fraction of games hitting the turn cap.

Definition of done (Phase 13)

- The loop runs many iterations without crashing or leaking memory.
- Metrics are logged and plottable; checkpoints save and reload identically.

14. Monte Carlo Tree Search and AlphaZero for Catan

You asked whether there is value in Monte Carlo Tree Search. The short answer: **yes, but with real caveats specific to Catan**, and it is best added *after* you have a working network and a trustworthy engine. This section explains what MCTS is, why the AlphaZero recipe is so effective, the three features of Catan that complicate it, and a concrete recommendation for sequencing it into your project.

14.1 What MCTS does

MCTS is a planning algorithm. Given the current position, it grows a search tree of possible futures and uses the statistics it gathers to pick a strong move. Each **simulation** repeats four steps:

1. **Selection**: from the root, walk down the tree choosing children that balance high estimated value (exploitation) with low visit counts (exploration), via a formula like UCT/PUCT.
2. **Expansion**: at a leaf, add one or more child nodes for legal actions (using your engine's legal_actions / apply_action).
3. **Evaluation**: estimate how good the leaf is - classically by a random rollout to the end of the game, in AlphaZero by a neural-network value estimate.
4. **Backpropagation**: push that estimate back up the path, updating visit counts and value averages.

After a fixed budget of simulations, the move with the highest visit count at the root is played. More simulations means stronger play, at linear cost.

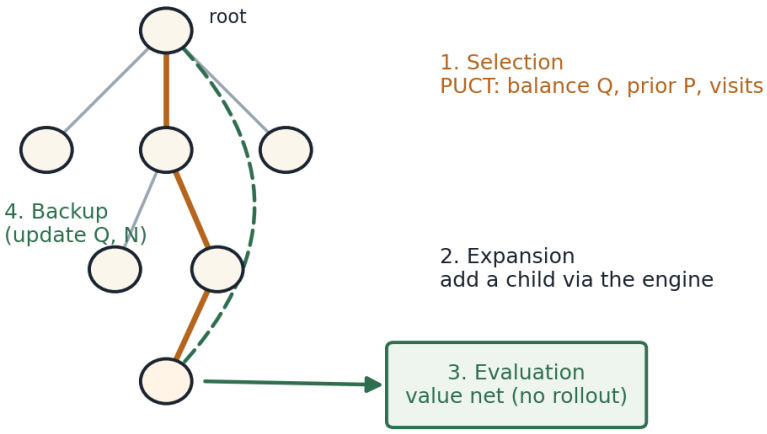


Figure 14.1 - One MCTS simulation. Selection walks down by PUCT (orange path), expansion adds a leaf via the engine, the value network evaluates it (replacing random rollouts in AlphaZero), and the result is backed up to the root.

14.2 The AlphaZero recipe

AlphaZero replaces the two weakest parts of classical MCTS with a neural network - the same actor-critic network you already built:

- The **policy** head provides a *prior* over actions, steering selection toward plausible moves (the PUCT formula), so the search does not waste simulations on obviously bad moves.
- The **value** head replaces random rollouts with a single evaluation, giving a far less noisy leaf estimate and removing the need to simulate to the end of the game.

Training is a loop: play games where each move is chosen by MCTS; record, for each position, the search's visit-count distribution (a better target than the raw policy) and the eventual game outcome; train the network to predict those; repeat with the improved network. Search makes the network better, and the better network makes search stronger - a virtuous cycle. Crucially, this reuses your representation and network unchanged; only the data-generation and target differ from PPO.

Tip Your investment in representation (Sections 6-10) pays off twice: the very same GNN with factored policy/value heads is exactly what an AlphaZero agent needs. Good representation is algorithm-agnostic.

14.3 Why Catan is harder for MCTS than Chess or Go

Go and Chess are deterministic, perfect-information, two-player, zero-sum games - the ideal habitat for MCTS. Catan breaks all three of those assumptions, and each break needs handling.

Challenge 1 - Stochasticity (the dice)

Actions do not lead to a single next state; a dice roll intervenes. The tree must model this with **chance nodes**: after a "roll" action, branch over the possible outcomes weighted by their probabilities (expectimax-style), or sample an outcome per simulation and rely on many simulations to average out. Chance nodes widen the tree and dilute the value of deep search, because you cannot count on a particular future.

Challenge 2 - Hidden information

Opponents' hands, the dev-card deck, and the random robber steal are unknown. Standard MCTS assumes you can perfectly simulate forward, which you cannot. Two established fixes:

- **Determinization**: sample a concrete "guess" of the hidden information (a plausible set of opponent hands consistent with what you have observed), run normal MCTS on that fully-observable world, and repeat over many sampled worlds, averaging the results. Simple and surprisingly effective.
- **Information-Set MCTS (ISMCTS)**: build the tree over information sets (everything you cannot distinguish) rather than exact states, sampling a determinization per simulation. More principled, more work to implement.

Challenge 3 - More than two players

AlphaZero's clean min/max logic assumes a two-player zero-sum game. Catan has four players. The standard adaptation is to make the value head output a **vector** - one predicted outcome per player - and have each node maximize the *acting* player's own component during selection and backup (the 'max-n' approach). This is well understood but adds bookkeeping, and four-player dynamics (temporary alliances, kingmaking) are genuinely harder to value than a duel.

14.4 Cost reality on a laptop

MCTS is far more expensive than PPO at decision time: every move costs hundreds of simulations, and every simulation costs an engine step and (in AlphaZero) a network evaluation. On a CPU this is slow. Mitigations: keep the network small, use a modest simulation budget (even 50-200 sims helps), cache network evaluations, and exploit your fast copy-or-undo state model from Section 3.3. Full AlphaZero self-play training is realistically a Colab-GPU activity (Section 18); test-time search (below) is feasible on CPU.

14.5 The pragmatic recommendation

Do not start with MCTS. Sequence it like this:

1. **Phase A - PPO first.** Train the network with PPO self-play (Sections 12-13). This validates your engine, representation, masking, and reward, and yields a usable policy and value function. It is the cheapest path to a working agent.
2. **Phase B - MCTS at test time.** Keep the PPO-trained network frozen and wrap it in MCTS only when *playing* (evaluation or against humans). The policy prior plus value estimate make even a few hundred simulations noticeably stronger than the raw policy - especially for high-stakes,

tactical decisions like initial placement and robber moves. This is a large strength gain for modest extra code and no retraining.

3 **Phase C - full AlphaZero (ambitious).** If you want to push further, switch data generation to MCTS and train the network on visit-count and outcome targets, handling stochasticity (chance nodes), hidden information (determinization), and four players (value vector). Expect this to need a GPU and considerable patience.

In other words: PPO buys you a working agent quickly; test-time MCTS is a high-value, low-risk add-on; full AlphaZero is a powerful but heavy final upgrade. The representation work you did earlier is what makes all three share the same network.

14.6 PUCT in one expression (for Phase B/C)

```
# choose the child maximizing:  Q(s,a) + c_puct * P(s,a) * sqrt(N(s)) / (1 + N(s,a))
#   Q(s,a)   = mean value backed up through action a
#   P(s,a)   = network's prior probability for a (masked + renormalized)
#   N(s)     = visits to parent;  N(s,a) = visits to this child
#   c_puct   = exploration constant (~1.0-2.5, tune it)
```

Tip Add Dirichlet noise to the root prior during AlphaZero self-play to force exploration; omit it at evaluation time so the agent plays its best.

Definition of done (Phase 14, if pursued)

- A test-time MCTS that consumes the frozen network and beats the raw policy head-to-head.
- Chance nodes for dice and determinization for hidden info, if you go past Phase B.
- A vector value head (one entry per player) if you attempt full multi-player AlphaZero.

15. Evaluation and Diagnostics

Training metrics can lie (a rising reward might just mean the opponent got worse). Evaluate against **fixed** opponents so the yardstick never moves.

15.1 The baseline ladder

1 **Random baseline**: learner vs three uniform-random agents. The chance baseline is 25%. Your first milestone is beating it convincingly (60-70%+). If you cannot beat random, hunt the bug - do not tune hyperparameters.

2 **Greedy heuristic baseline**: a hand-written bot (build the best thing affordable, place on high-probability tiles). Beating it shows real strategy.

3 **Past selves**: win rate vs older checkpoints should trend upward, confirming genuine improvement rather than rock-paper-scissors cycling.

4 **Policy vs policy+MCTS**: if you built Section 14, confirm search beats the raw network - a direct check that your MCTS helps.

15.2 Make evaluation statistically honest

- Play hundreds of games and report a confidence interval - Catan is high-variance.
- **Rotate seats**: the first player has an edge, so evaluate in every seat equally.
- Use a **held-out set of seeds** never seen in training.
- Report both **sampled** play (training distribution) and **greedy/argmax** play (peak skill).

15.3 Watch games

Build a simple text or matplotlib renderer that replays a game move by move. Nothing surfaces bugs and bad strategies faster. Watch for pathologies: never building cities, hoarding cards, endless bank trades, or nonsensical robber moves.

Definition of done (Phase 15)

- A repeatable script reports win rate with confidence intervals vs each baseline.
- Seat rotation and held-out seeds are in place; you can render any single game.

16. Iteration and Reward Shaping

Once the loop works and beats random, the project becomes a cycle of small, measured improvements. Change **one thing at a time** and compare against fixed baselines, or you will not know what helped.

16.1 Reward shaping to fight sparsity

Pure win/loss is honest but slow to learn from. Shaping adds small intermediate rewards that point roughly the right way, then is annealed away so the agent ultimately optimizes the true objective.

- Small positive reward per victory point gained (a strong, natural Catan signal).
- Tiny rewards for productive milestones: a new settlement/city, access to a new resource or a high-probability tile.
- Keep shaping **small** relative to the +1 win, and anneal it toward zero as training matures.

Tip Beware reward hacking: if you reward bank trades, the agent may trade pointlessly to farm reward. Shape **outcomes you value** (VP, board position), not **actions** you merely hope are useful.

16.2 A tuning checklist when learning stalls

- Entropy collapsed early -> raise entropy coef or lower learning rate.

- Unstable / loss spikes -> lower clip epsilon or learning rate, clip gradients.
- Critic not learning (low explained variance) -> raise value coef, check value targets.
- Agent ignores an action type -> verify the mask exposes it and the encoding is correct.
- No improvement at all -> suspect a bug in reward assignment, masking, or encoding before touching hyperparameters.

16.3 Keep an experiment log

Record each run's hyperparameters, code version (git commit), representation variant, and resulting baseline win rates. RL is noisy; without a log you will repeat experiments and draw false conclusions. A spreadsheet or TensorBoard/Weights & Biases is enough.

17. Expanding the Rules

With a working learner on the minimal game, add complexity one rule at a time. After each addition, re-run validation (Section 5) and re-evaluate. Expanding rules means extending three things in lockstep: the engine, the representation, and (if used) the search.

17.1 Suggested order

1. **Ports** (2:1 / 3:1 trades). Small engine change; add trade actions and port access to vertex features. A gentle first expansion.
2. **Development cards** (knight, road building, year of plenty, monopoly, VP). Adds a deck, hidden cards, and the knight/robber interaction; also enables **largest army**.
3. **Longest road**. No new actions, but the longest-trail computation over a player's road network is genuinely tricky. Add it once you are ready.
4. **Player-to-player trading**. The hardest by far: it explodes the action space and turns the game into a negotiation. Start with a drastically simplified trade interface.

17.2 The recurring pattern for each new rule

1. Extend GameState with any new data (deck, players' dev cards, etc.).
2. Extend legal_actions / apply_action; add unit tests.
3. Add new action indices/heads and update the mask builder.
4. Add the new information to the feature groups (Section 7) and graph (Section 8).
5. Bump network input/output dims (usually retrain from scratch when these change).
6. Re-validate with random games, then retrain and re-evaluate.

Tip Every time the observation or action dimensions change, old checkpoints become incompatible. Version your environment (CatanEnv-v1, v2, ...) so you never load a mismatched checkpoint.

18. Scaling Up (GPU / Colab)

On a laptop CPU you can get the minimal game learning, but rollouts will be the bottleneck because the engine runs in Python. Attack the bottleneck before reaching for a GPU.

18.1 Speed up rollouts first

- **Profile** a training iteration. The 54-node network is tiny; the simulator almost always dominates, so a GPU sits idle if rollouts are slow.
- **Vectorize environments**: run many games in parallel (multiprocessing) and batch their observations into one network forward pass.
- Optimize engine hot paths; switch to in-place mutation with undo (Section 3.3) if copying dominates. This also directly speeds up MCTS.

18.2 When the GPU actually helps

- Once the network is larger (deeper GNN, attention) and you batch many parallel envs per forward pass, a GPU pays off. Colab is convenient here.
- Move the model and batched tensors to the GPU; keep the simulator on CPU workers and transfer only batched observations.
- Colab sessions are ephemeral - checkpoint frequently to Google Drive so a disconnect does not lose hours.
- AlphaZero self-play (Section 14, Phase C) is the case that most needs a GPU, because it evaluates the network thousands of times per move.

18.3 Smarter encodings revisited

If you began with the MLP, the GPU era is the natural time to commit fully to the GNN and to richer message passing (attention over neighbors, the heterogeneous tile/vertex/edge graph). These respect Catan's structure and typically learn faster - the representation investment compounds as you scale.

Appendix A: Reference Tables

A.1 Board composition (base game)

Resource	Tiles	Terrain
Wood	4	Forest
Sheep	4	Pasture
Wheat	4	Field
Brick	3	Hills
Ore	3	Mountains
Desert	1	Robber starts here; produces nothing

Number tokens overall: one each of 2 and 12; two each of 3, 4, 5, 6, 8, 9, 10, 11 - that is 18 tokens for the 18 non-desert tiles. There is no 7; rolling a 7 triggers the robber.

A.2 Building costs, victory points, and supply

Item	Cost	VP	Per-player supply
Road	1 wood, 1 brick	0	15
Settlement	1 wood, 1 brick, 1 sheep, 1 wheat	1	5
City	2 wheat, 3 ore	2	4
Dev card (later)	1 sheep, 1 wheat, 1 ore	varies	deck of 25

A.3 Dice roll probabilities (encode tokens by this, not face value)

Roll	2	3	4	5	6	7	8	9	10	11	12
Ways/36	1	2	3	4	5	6	5	4	3	2	1

A.4 Representation cheat-sheet

Decision	Recommended choice
Player encoding	Relative to current player (me / opp1 / opp2 / opp3 / none)
Board encoding (v1)	Flat normalized feature vector + MLP
Board encoding (v2)	Vertex graph + GNN with factored policy heads
Number tokens	Dice probability, not face value
Action space	Fixed index space + boolean mask, applied to logits pre-softmax
Policy head	Per-node/edge/tile heads tied to graph embeddings

Augmentation	12 board symmetries x 4 seat rotations
Search	PPO first; test-time MCTS next; full AlphaZero last

Appendix B: Pitfalls Checklist

Engine

- Geometry tables not validated -> impossible bugs. Validate before trusting them.
- Forgetting the desert has no number and starts with the robber.
- Distance / connectivity rules applied inconsistently between setup and main phases.
- Resource conservation broken (cards created or destroyed). Assert it constantly.
- Randomness not routed through one seeded RNG -> non-reproducible bugs (and broken determinization for MCTS).

Representation

- Absolute seat encoding -> the shared policy cannot generalize across seats.
- Raw integers (counts, face-value tokens) fed unnormalized -> slow/unstable learning.
- Action index mapping disagreeing between engine, mask, and network.
- Symmetry augmentation that permutes features but not the action target -> mislabeled data, worse than no augmentation.
- Graph adjacency that does not match the validated geometry tables.

RL and search

- Masking applied after softmax instead of to the logits -> probability leaks to illegal moves.
- Reward assigned to the wrong seat's trajectory.
- Truncated games treated as true terminals instead of bootstrapped.
- Entropy bonus too small -> premature collapse to one action.
- Reaching for MCTS or fancy algorithms before beating the random baseline with PPO.
- MCTS without chance nodes for dice, or without determinization for hidden info -> systematically wrong search.

Appendix C: Glossary and Resources

Mini-glossary

Term	Definition
Action mask	Boolean vector marking which fixed action indices are legal now
On-policy	Learning only from data generated by the current policy (PPO is on-policy)
GAE	Generalized Advantage Estimation; smooths advantages via gamma and lambda
Entropy bonus	A term rewarding a less-certain policy, encouraging exploration
Self-play	Training an agent by having it play against copies of itself
Bootstrapping	Using the critic's value estimate in place of unknown future reward
GNN	Graph neural network; learns by passing messages along graph edges
Equivariance	Relabeling the input relabels the output the same way (e.g. node order)
MCTS	Monte Carlo Tree Search; plans by simulating many futures from the root
PUCT	The selection rule MCTS uses to balance value, prior, and visit counts
Determinization	Sampling hidden info into a concrete world so MCTS can plan on it
ISMCTS	Information-Set MCTS; searches over indistinguishable states under hidden info
Max-n	Multiplayer search where each node maximizes the acting player's value entry
Chance node	A tree node modeling a random event (a dice roll) with weighted outcomes

Suggested study resources

- **Red Blob Games - Hexagonal Grids**: the definitive reference for hex coordinate math; use it for geometry and for deriving the board symmetries.
- **The PPO paper** (Schulman et al.) and the blog post 'The 37 Implementation Details of PPO' for the practical gotchas.
- **Spinning Up in Deep RL** (OpenAI): clear, implementation-focused RL explanations.
- **AlphaGo Zero / AlphaZero papers** (Silver et al.) for the search-plus-network recipe.
- **Information Set Monte Carlo Tree Search** (Cowling, Powley, Whitehouse) for hidden information.
- **PyTorch Geometric** / **DGL** docs for GNNs; **catanatron** (open-source Python Catan) for engine and representation ideas - learn from it, write your own.

Final advice

Build bottom-up, verify each layer before adding the next, and treat representation as a first-class design problem rather than an afterthought - it is the lever with the most leverage. Get a clean engine,

encode the game as a graph with relative, normalized features and graph-tied action heads, train it with PPO until it convincingly beats random, and only then consider tree search. Add MCTS first as a test-time booster on your trained network; reach for full AlphaZero only when you have the patience and a GPU. Do that, and you will end up with an agent that genuinely learns to play Catan - and a deep, hard-won understanding of how modern game-playing AI fits together.

End of guide - now go build it.